Coding and Testing, part-II

S. S Satapathy

Functional vs. Structural testing

- **Functional** or **black-box** testing
  - test cases are designed using only the functional specification of the software, i.e. without any knowledge of the internal structure of the software.
- **Structural** or **white-box** testing
  - designing test cases requires thorough knowledge about the internal structure of software.

Is the program testable?

- We **can test** a program, **if** we can **provide input** and **observe output**.
- **If** a **program** is **controllable** and **observable** **then** it is called **testable**.
  - If we can **define** the **input** and **ensure** that **appropriate part** of the program **that manipulate the input** is **reached**, then we call the program **is controllable**.
  - If we **can observe the output** of a program and evaluate the test, then we call the program is **observable**.

A model for detecting fault by observation

- **RIPR model**: Four conditions necessary for observing failure
  - **Reachability**
    - location of the faulty Code must be reachable during execution
  - **Infection**
    - State of the program must be incorrect
  - **Propagation**
    - The infected state must produce some incorrect result or final state
  - **Reveal**
    - The tester must observe part of the incorrect portion of the program state

A model for detecting fault by observation

```
input x,y;
If(x<10)
{
        z=x+1;
        if(y<z){
                Erroneous
                Code Block
        }
}
```

- RIPR model: Four conditions necessary for observing failure
  - **Reachability**: **TRUE**, any value of x allows to reach first condition and x<10 allows to reach 2nd if stmt.
  - **Infection**: if statements will be executed depending on values of x and y
  - **Propagation**: x<10 and y<z will allow to reach the error block and may lead to erroneous o/p

Model Based Testing

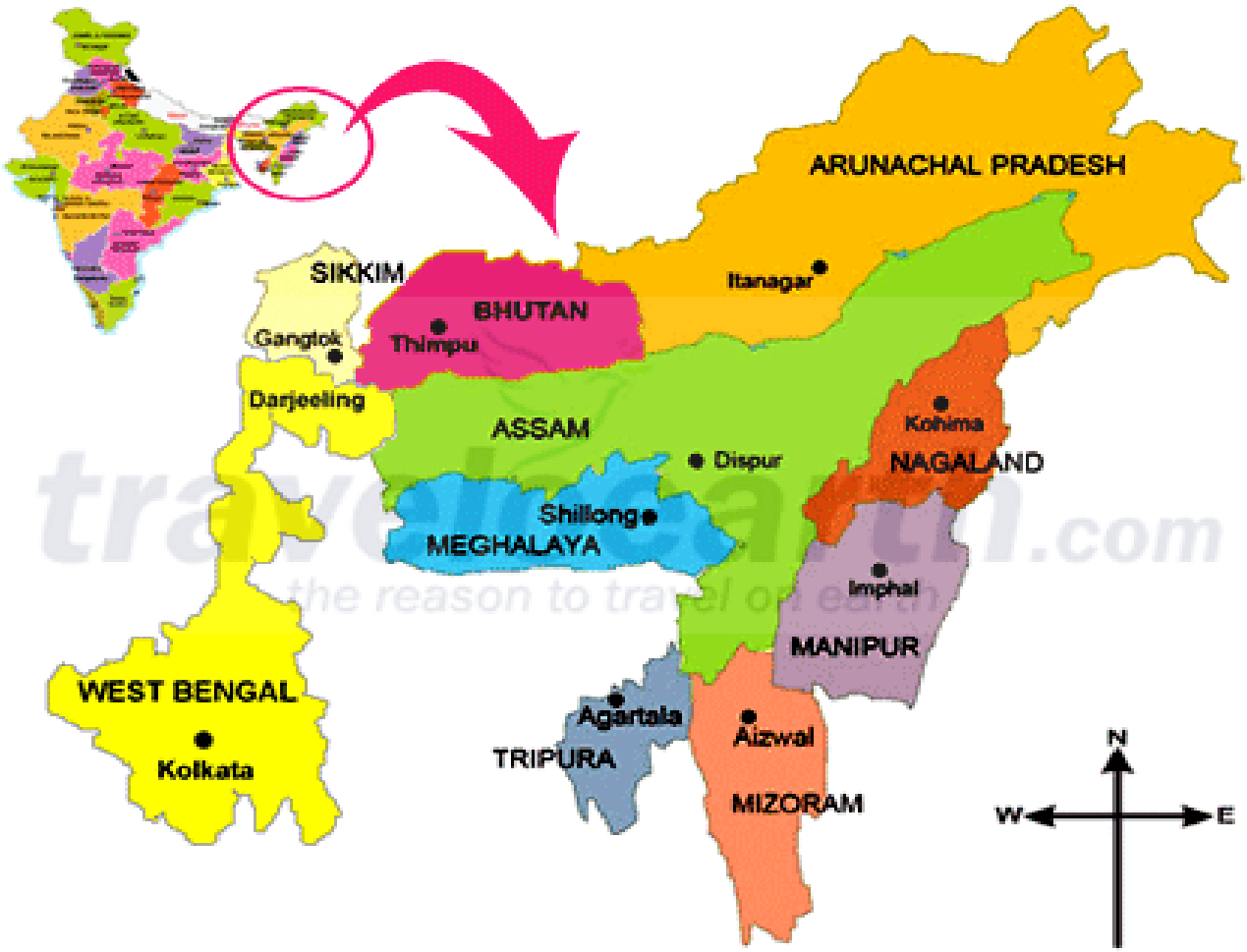
- **Model based testing involves working with a model of the software** and perform testing
  - A **model is an abstraction of the software** presenting some of the features
  - A model could based on the following:
    - **Formal or mathematical notations** such as formal language, graphs, logical formula etc.
    - **Modeling Language** such as UML
  - Models can also be used for Requirement analysis, design etc.
  - Testing with models for requirement and design may detect error early

Different Models/Structures Used

- Based on the **model of the software, coverage criteria**, and then **the test cases** are designed
  - **Graphs** can be useful to represent flow of **control in program and also flow of data in design**
  - Truth value of the **Logical expressions** **guide control flow in program execution** and also define conditions in requirements
  - **Sets** can be used to **specify input domain** in black box design
  - **Syntax** of programming language can be represented using **grammar**

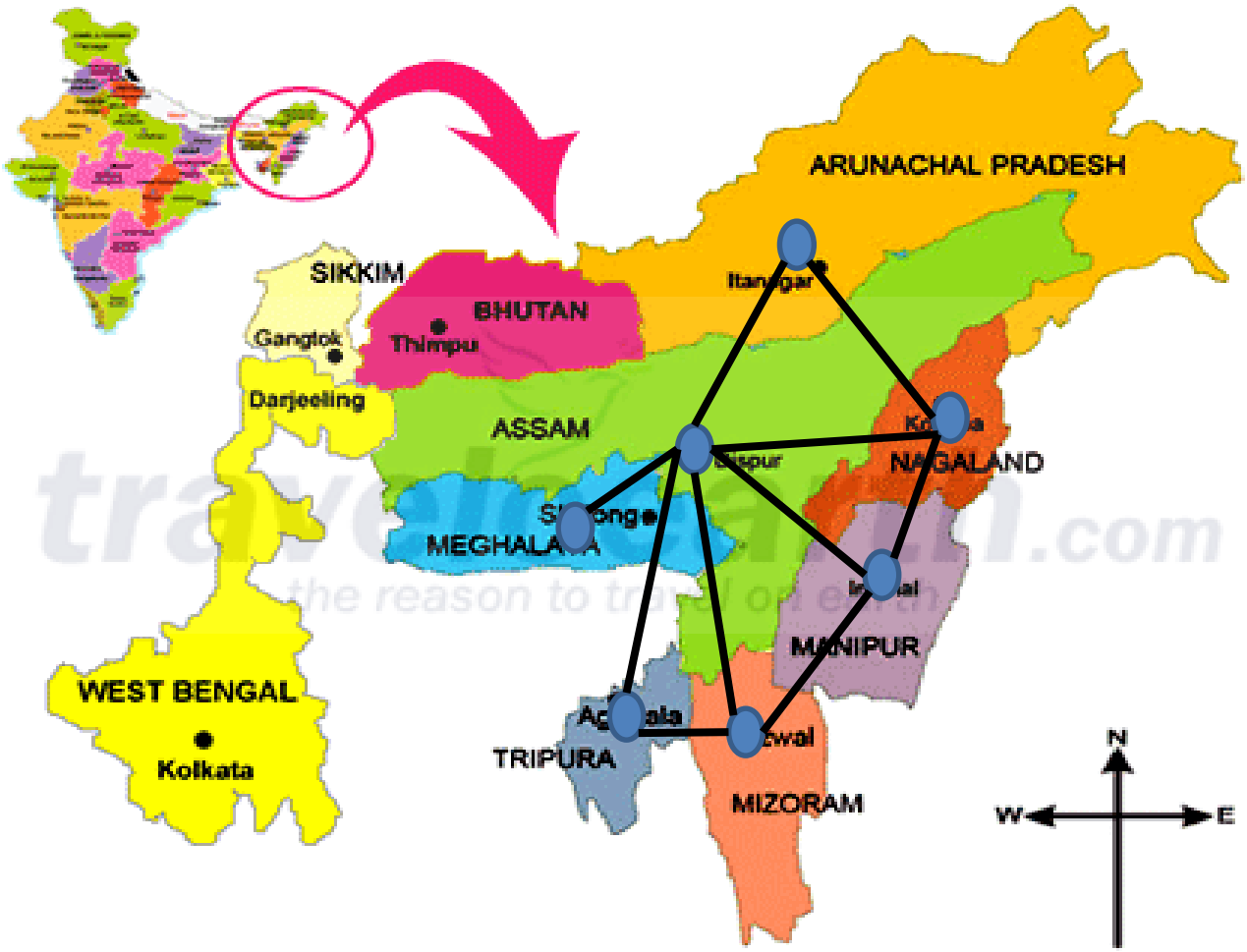
Graph based model for S/W testing

Introduction to Graph Theory

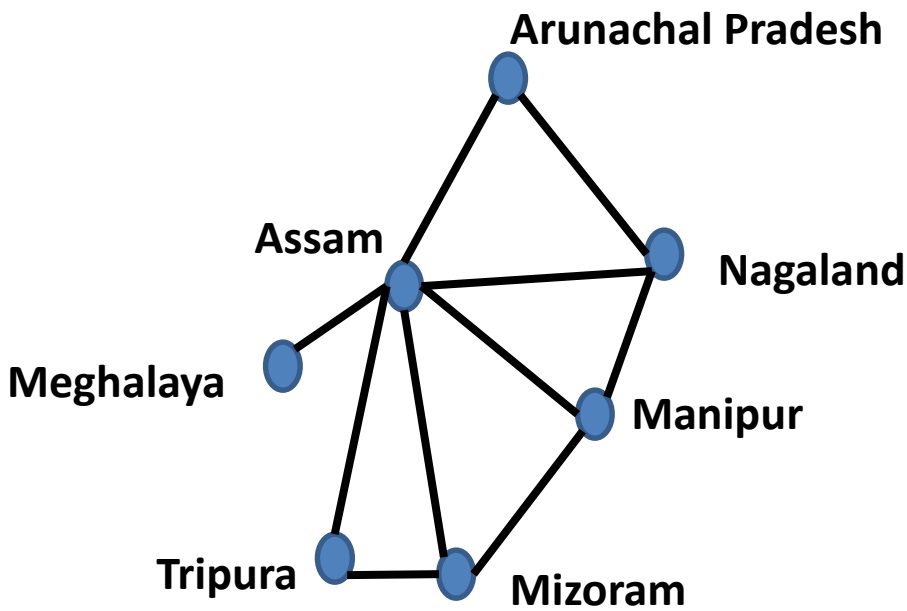


How to represent the Map in computer?

Introduction to Graphs



Introduction to Graphs



Graph for North-East India

A **vertex (Node)**: a region

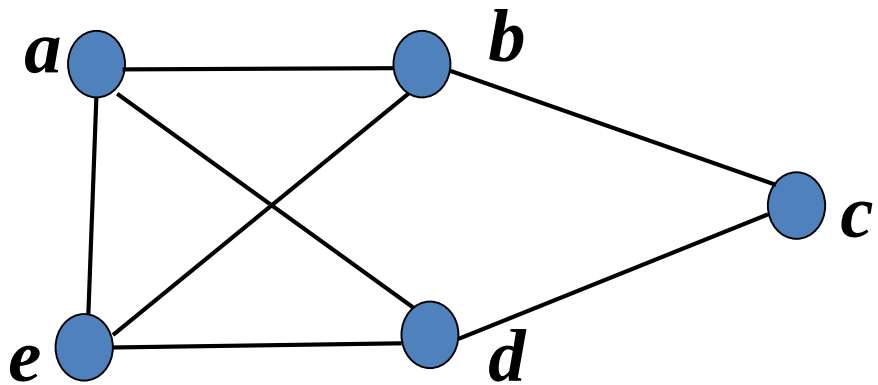
An **edge** : a edge(link) between two regions

A graph ***G*** is consisting of:

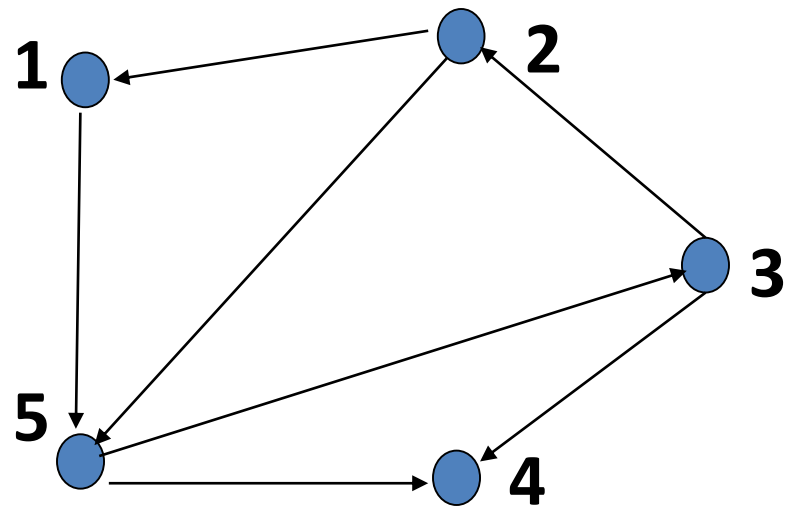
A vertex set ***V(G)***

An edge set ***E(G)***

Introduction to Graphs



A graph



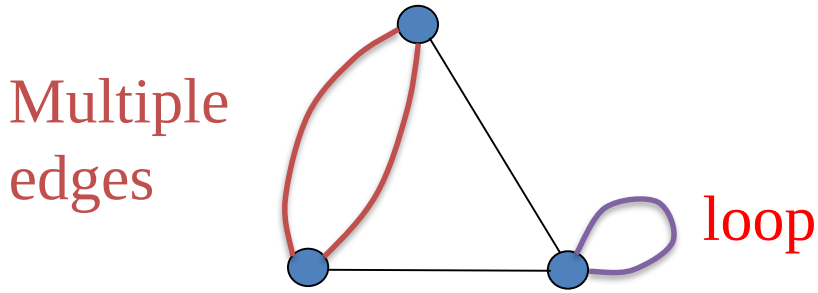
A diagraph

A **graph G** is consisting of: A **vertex** set V(G)

An **edge** set E(G)

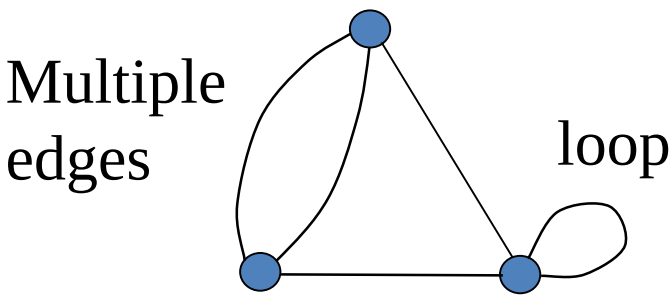
Loop, Multiple edges

- Loop : An edge whose endpoints are equal
- Multiple edges : Edges have the same pair of endpoints

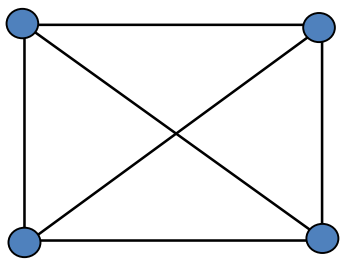


Simple Graph

- Simple graph : A graph has no loops or multiple edges



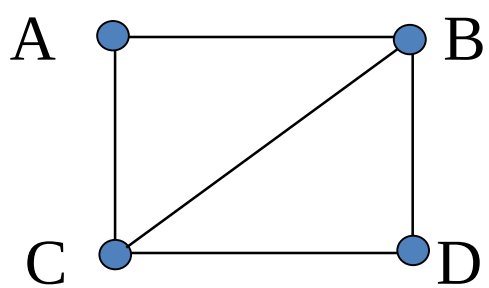
It is not simple.



It is a simple graph.

Adjacent, neighbors

- Two vertices are *adjacent* and are *neighbors* if they are the endpoints of an edge
- Example:
  - *A* and *B* are adjacent
  - *A* and *D* are not adjacent

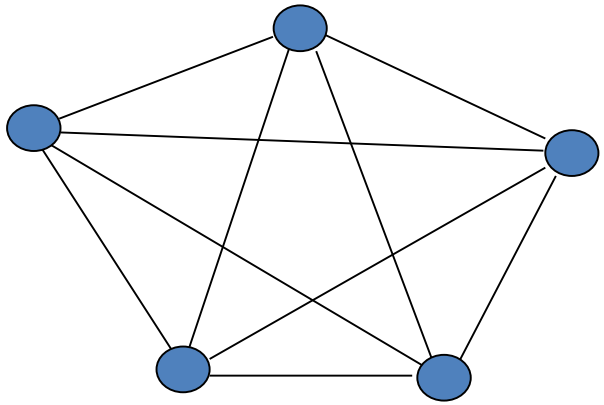


Finite Graph, Null Graph

- Finite graph : an graph whose vertex set and edge set are finite
- Null graph : the graph whose vertex set and edges are empty

Complete Graph

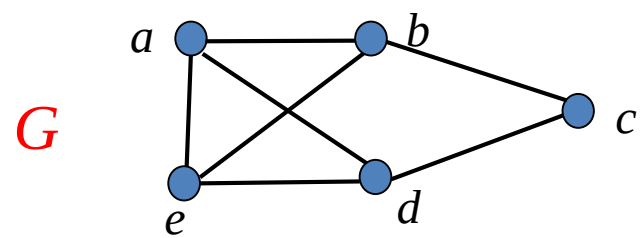
- **Complete Graph** : a simple graph whose vertices are pair-wise adjacent

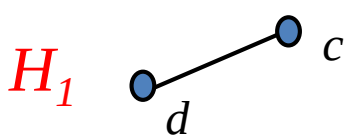


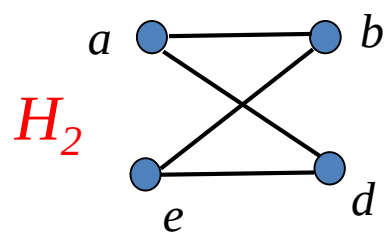
Complete Graph

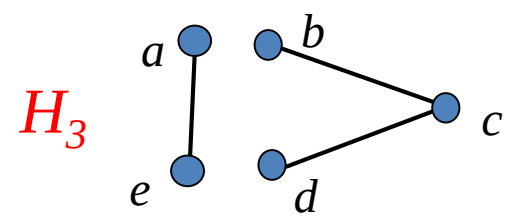
Subgraphs

- A subgraph of a graph ***G*** is a graph ***H*** such that:
  - $V(H) \subseteq V(G)$ and $E(H) \subseteq E(G)$ and
  - The assignment of endpoints to edges in ***H*** is the same as in ***G***.
- Example: H_1, H_2, and H_3 are subgraphs of ***G***



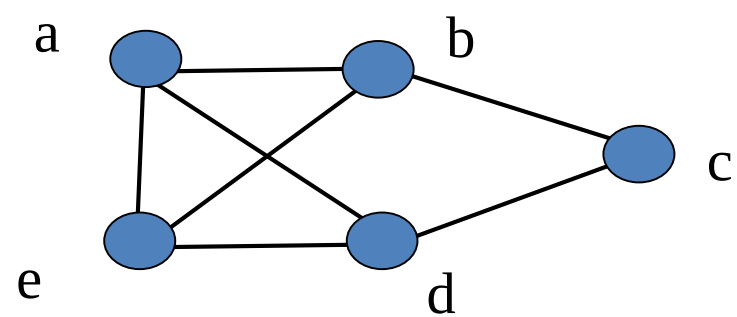






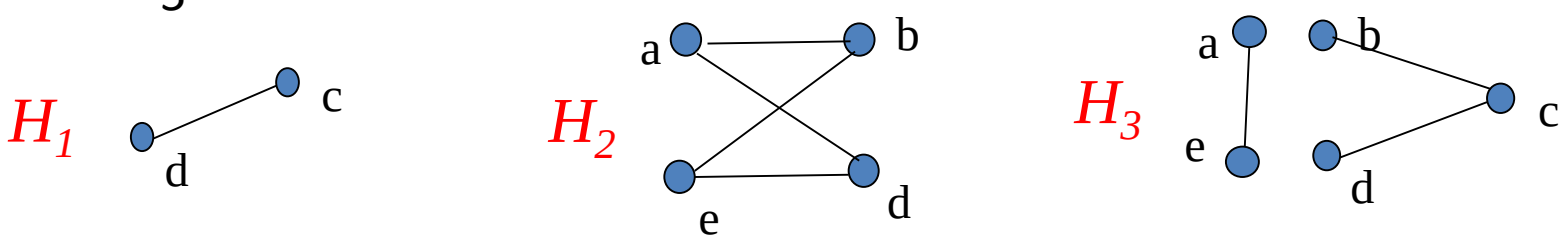
Path and Cycle

- Path : a sequence of distinct vertices such that two consecutive vertices are adjacent
  - Example: **(*a, d, c, b, e*)** is a path
  - **(*a, b, e, d, c, b, e, d*)** is not a path; it is a walk
- Cycle : a closed Path
  - Example: **(*a, d, c, b, e, a*)** is a cycle
- The length of a walk or, path, or cycle is its number of edges.



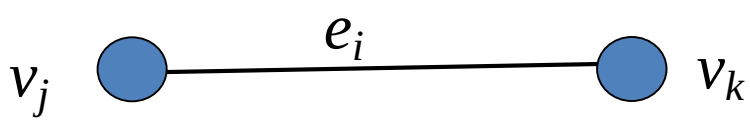
Connected and Disconnected

- Connected : There exists at least one path between any two vertices
- Disconnected : Otherwise
- Example:
  - H_1 and H_2 are connected
  - H_3 is disconnected



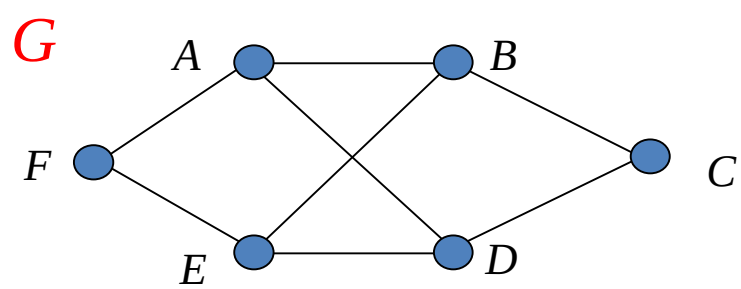
Adjacency, Incidence, and Degree

- Assume e_i is an edge whose endpoints are (v_j, v_k)
- The vertices v_j and v_k are said to be adjacent
- The edge e_i is said to be incident upon v_j
- Degree of a vertex v_k is the number of edges incident upon v_k. It is denoted as $d(v_k)$



Degree

- The **degree** of vertex v in a graph G, written or $d(v)$, is the number of edges incident to v, except that each loop at v counts twice
- The **maximal degree** is $\Delta(G)$
- The **minimum degree** is $\delta(G)$

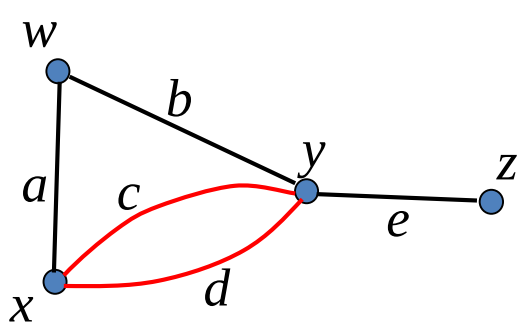


$d(B) = 3,\ d(C) = 2$

$\Delta(G) = 3,\ \delta(G) = 2$

Adjacency matrix

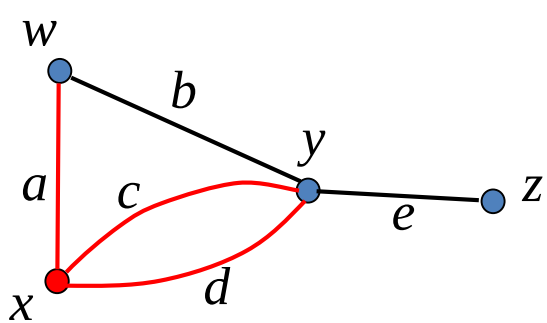
- Let $G = (V, E)$, $|V| = n$ and $|E|=m$
- The adjacency matrix of G written $A(G)$, is the n-by-n matrix in which entry $a_{i,j}$ is the number of edges in G with endpoints $\{v_i, v_j\}$.



$$\begin{array}{c c} & \begin{array}{cccc} w & x & y & z \end{array} \\ \begin{array}{c} w \\ x \\ y \\ z \end{array} & \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 2 & 0 \\ 1 & 2 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \end{array}$$

Incidence Matrix

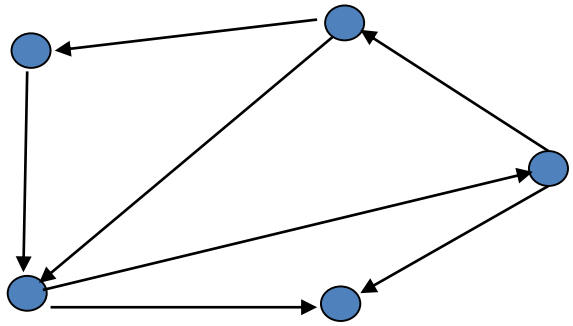
- Let $G = (V, E)$, $|V| = n$ and $|E|=m$
- The incidence matrix $M(G)$ is the n-by-m matrix in which entry $m_{i,j}$ is **1** if v_i is an endpoint of e_i and otherwise is **0**.



$$
\begin{array}{c c}
 & \begin{array}{ccccc} a & b & c & d & e \end{array} \\
\begin{array}{c} w \\ x \\ y \\ z \end{array} & \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}
\end{array}
$$

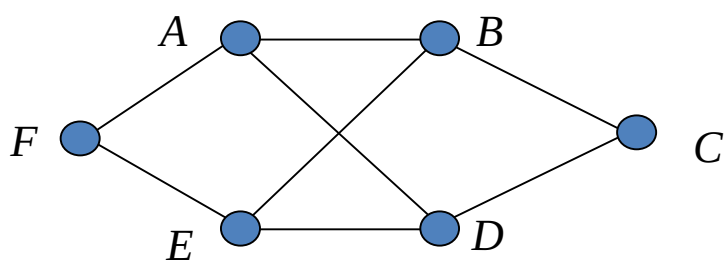
Directed Graph and Its edges

- A **directed graph** or **digraph** **G** is a triple:
  - A **vertex set** **V(G)**,
  - An **edge set** **E(G)**, and
  - A function assigning each edge an ordered pair of vertices.
    - The first vertex of the ordered pair is the **tail** of the edge
    - The second is the **head**
    - Together, they are the **endpoints**.
- An edge is said to be **from** its tail **to** its head.
  - The terms "head" and "tail" come from the arrows used to draw digraphs.



Graph Traversal

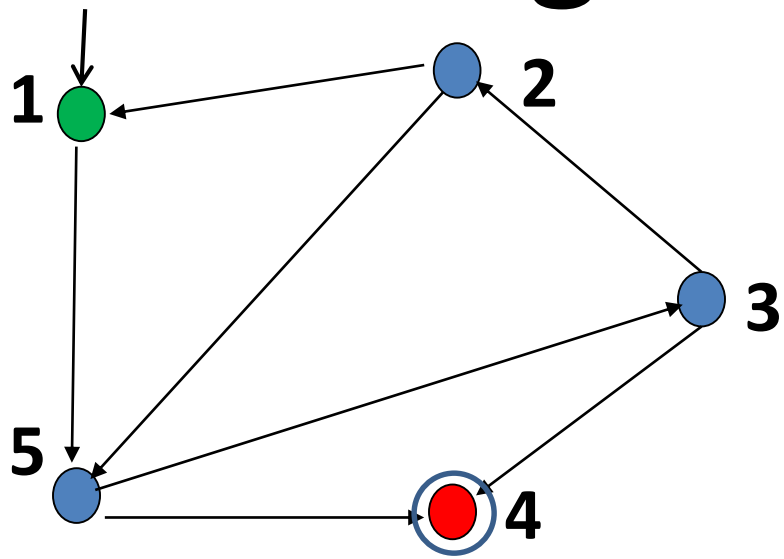
- Graph Traversal: Visiting all the nodes in a graph
  - DFS (Depth First Search)
  - BFS (Breadth First Search)



Graphs used in Software Testing

- **Graphs** are very much popular **for criteria based modeling of software** such as
  - **Control flow graph**
  - **Data flow graph**
  - **Call graph**
  - **Use case diagram**
  - **State chart diagram**
- **Nodes and edges** of these graphs are **labeled to represent different features** of the software
- **Testing of the software is to cover the graph**

Graphs used in Software Testing



Graph can be used as a model to present some of the features of a s/w. In this graph, there are two special nodes: one start node and one or more stop node(s), start node represents initiation of computation and stop nodes represent end of computation.

Control Flow Graph (CFG)

A graph G is consisting of:

A **vertex** set V(G)

An **edge** set E(G)

A **vertex (Node)**: a statement/ a sequence of statements

An **edge** : an edge(link) control flow

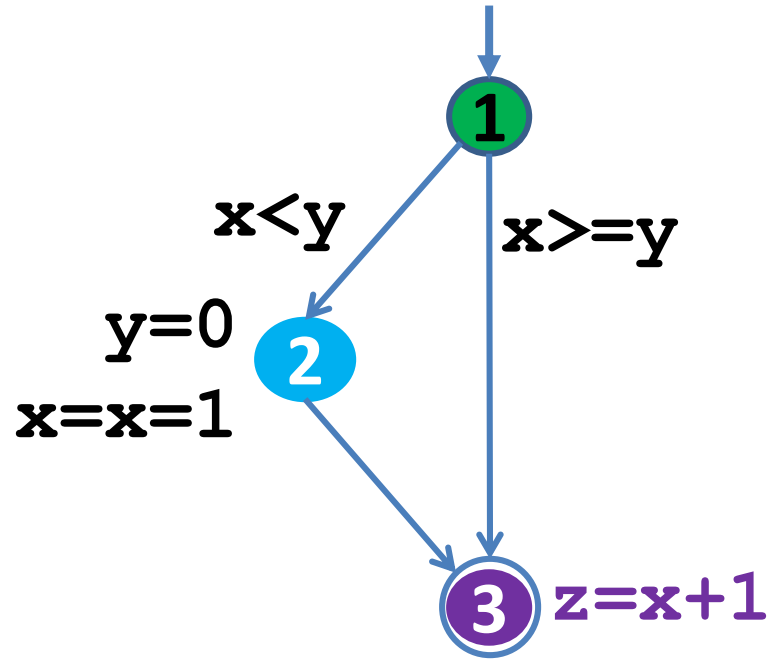
```
a=10;
    .
    .
    .
b=a+20;
```

A sequence of statements represented by a graph with a single node

Graphical representation (a graph with a single node)

CFG: If Statement

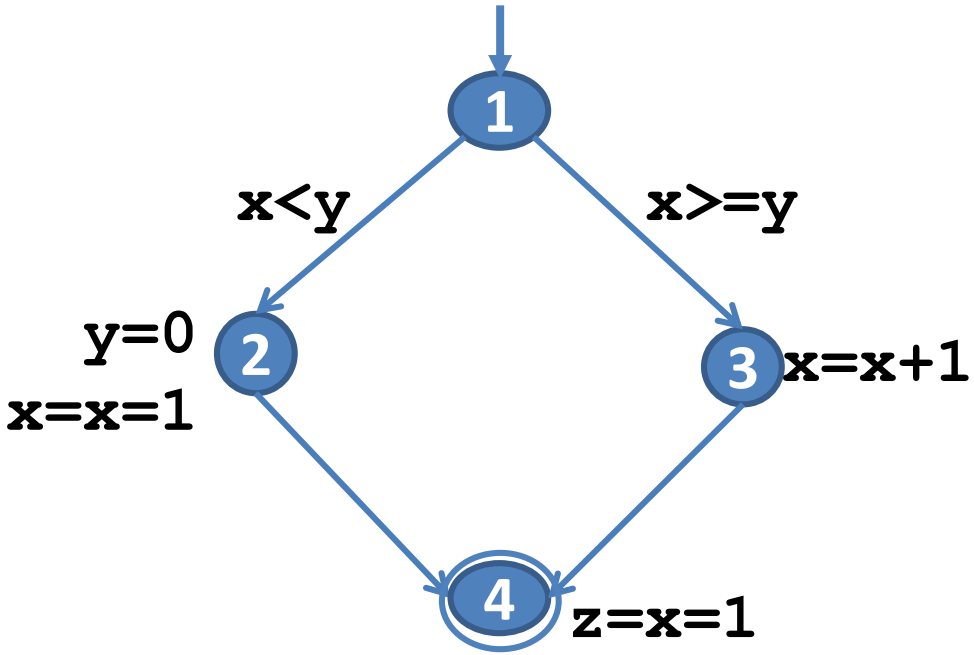
```
if(x<y){
      y=0;
      x=x+1;
}
z=x+1;
```



Note: Node 1 is a dummy node from which two edges are generated. The two edges represent two possible scenarios for the condition (x<y) and (x>=y) respectively.

CFG: If – Else Statement

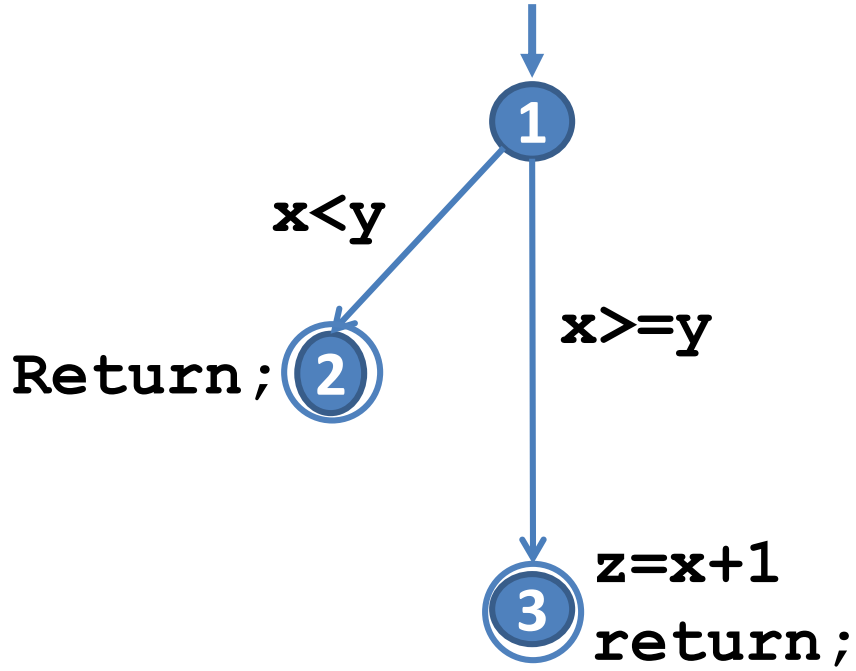
```
if(x<y){
      y=0;
      x=x+1;
}
else{
      x=x+1;
}
z=x+1;
```



Note: Node 1 is a dummy node from which two edges are generated. The two edges represent two possible scenarios for the condition (x<y) and (x>=y) respectively.

CFG: If Statement with return

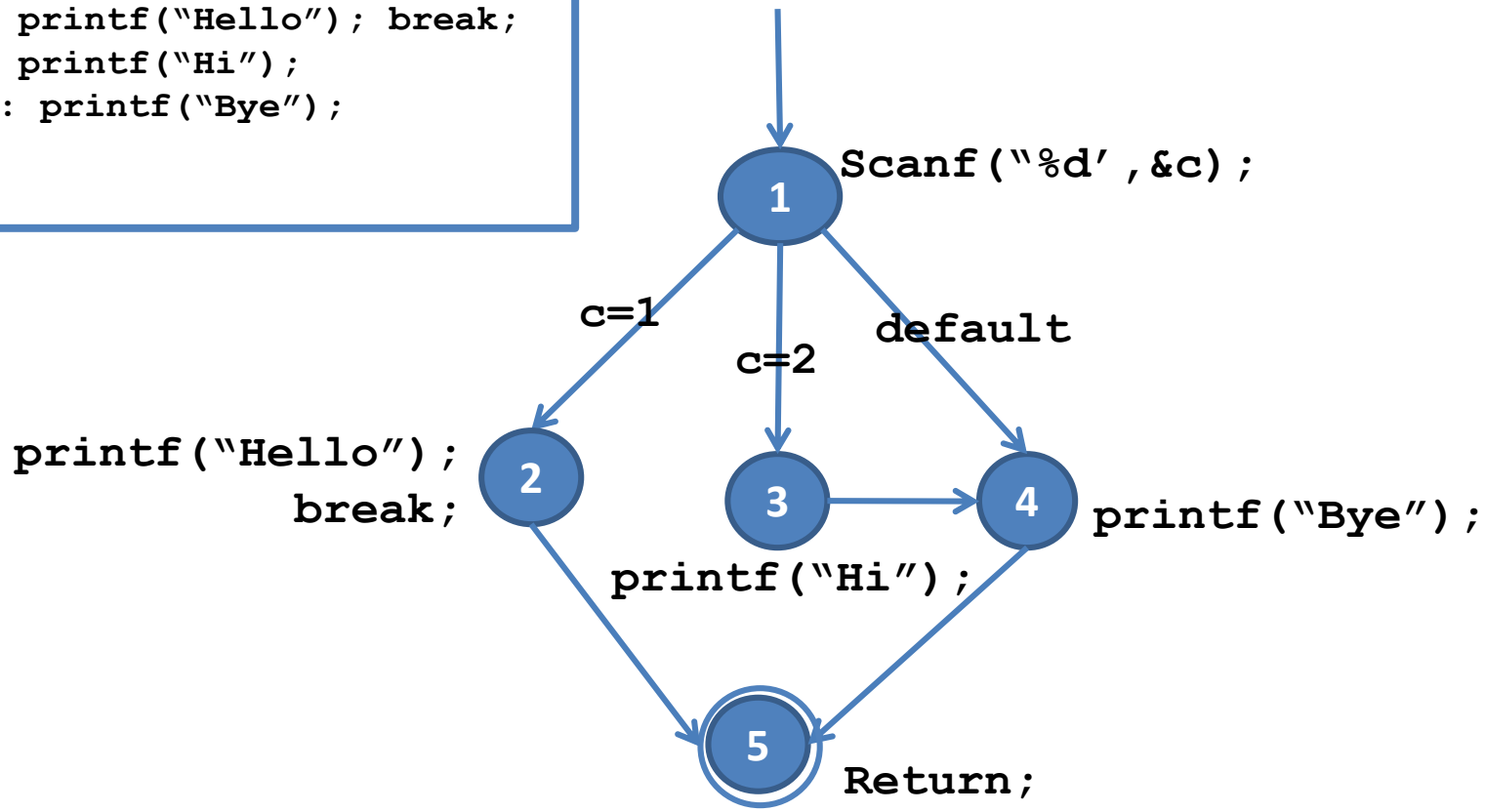
```
if(x<y){
      return;
}
z=x+1;
return;
```



Note: There are two possible termination of the program represented by two end nodes.

CFG: Switch Case Statement

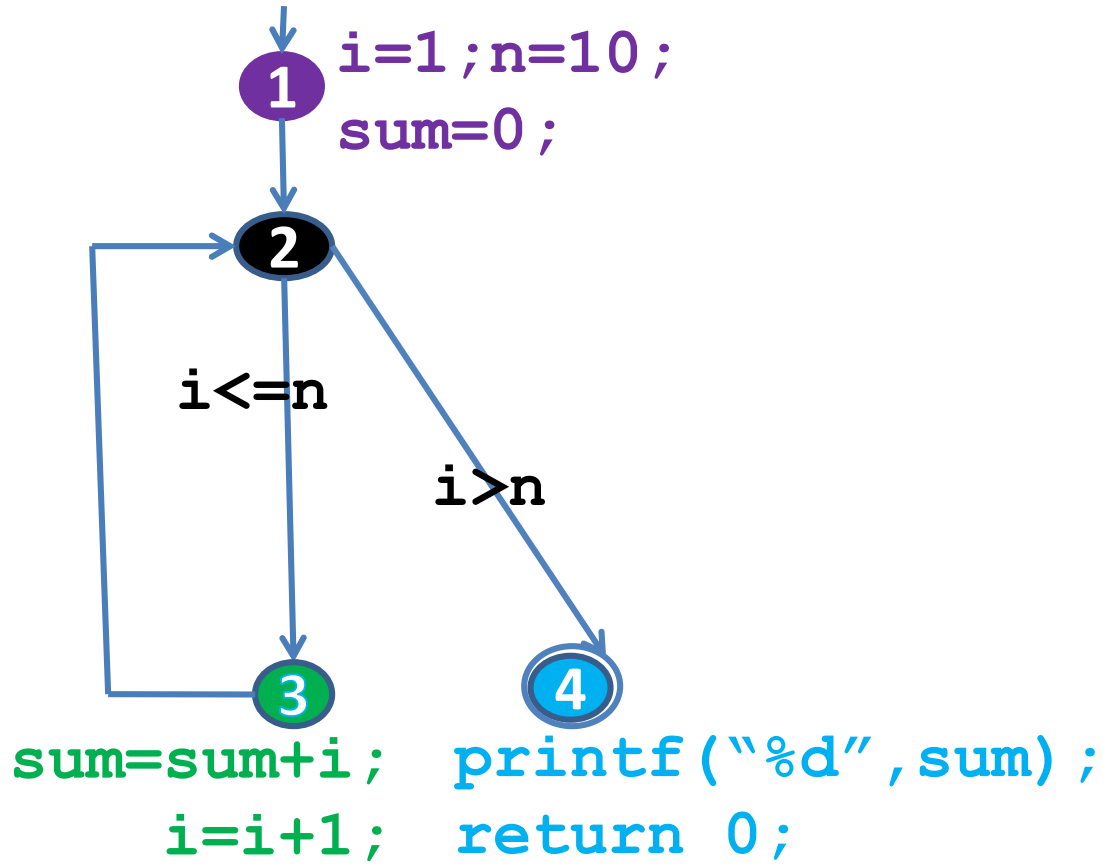
```
Scanf("%d',&c);
switch(c){
        case 1: printf("Hello"); break;
        case 2: printf("Hi");
        default: printf("Bye");
}
return;
```



Note: The edges in the CGF are carefully chosen to represent execution of the cases.

CFG: While Loop

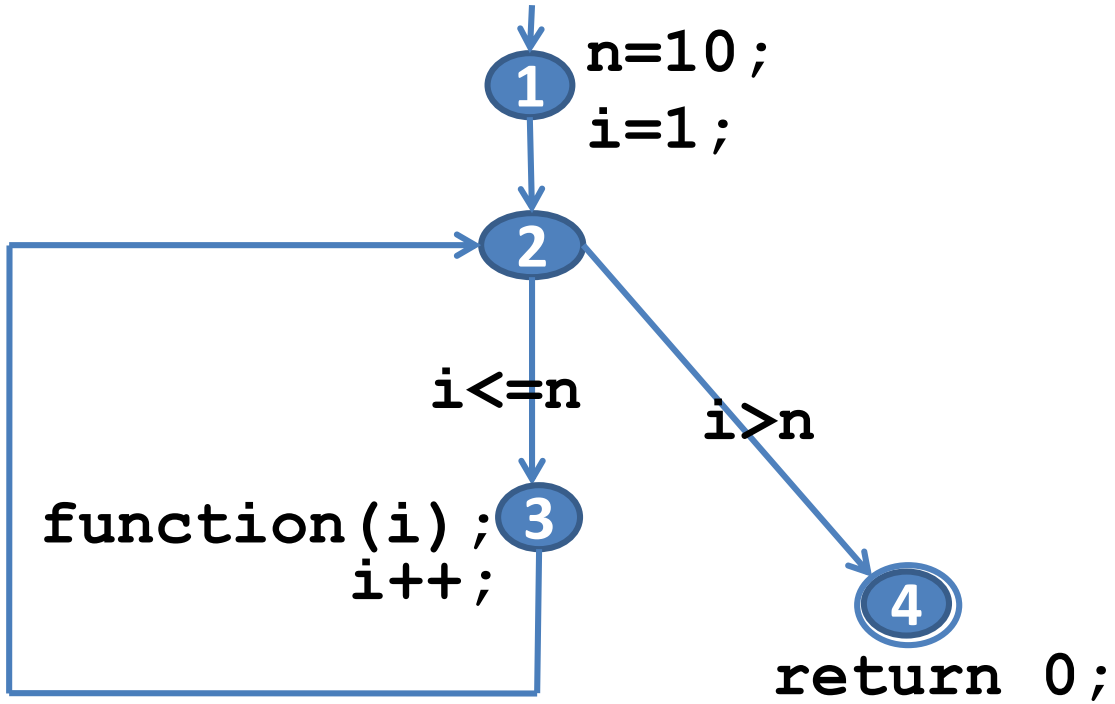
```
i=1;
n=10;
sum=0;
while (i<=n){
      sum=sum+i;
      i=i+1;
}
printf("%d",sum);
return 0;
```



Note: Node 2 is a dummy node from which two edges are generated. The two edges represent two possible scenarios for the condition (i<=n) and (i>n) respectively. Node 3 represents body of the loop, after execution of the statements in node 3, control flows to node 2 for next iteration.

CFG: For Loop

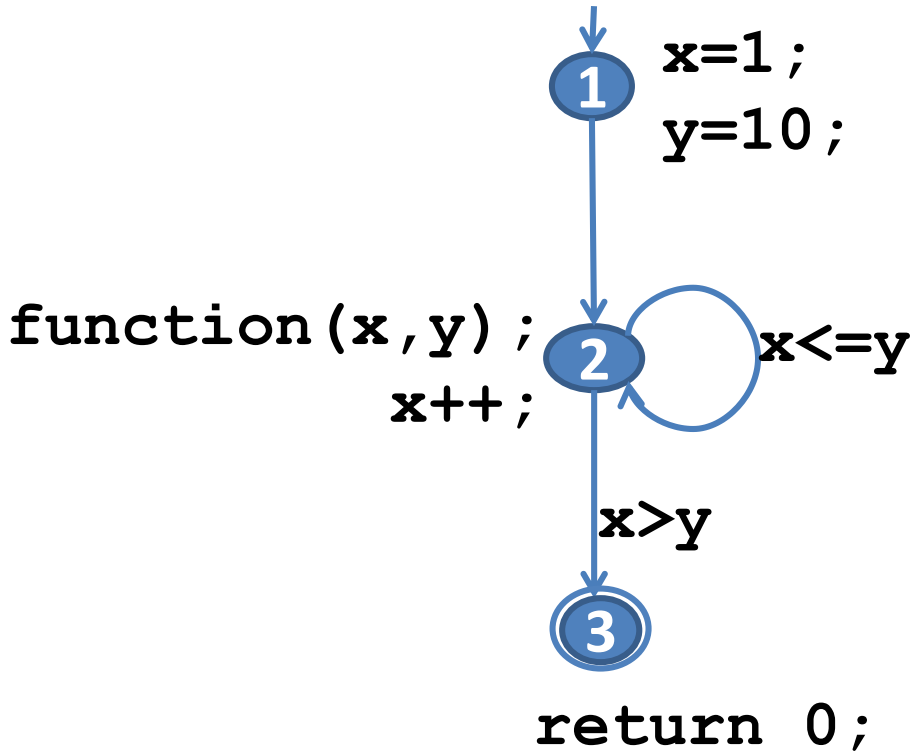
```
n=10;
for(i=1;i<=n;i++){
      function(i);
}
return 0;
```



Note: CFG is same to that of while loop

CFG: do-while Loop

```
x=1;
y=10;
do{
      function(x,y);
      x++;
}while(x<=y);
return 0;
```



Note: Node 2 represents body of the do-while loop. When the condition of the while statement is satisfied, the node 2 is again executed, tis is represented by the loop in the node 2.

Complete Examples of CFG

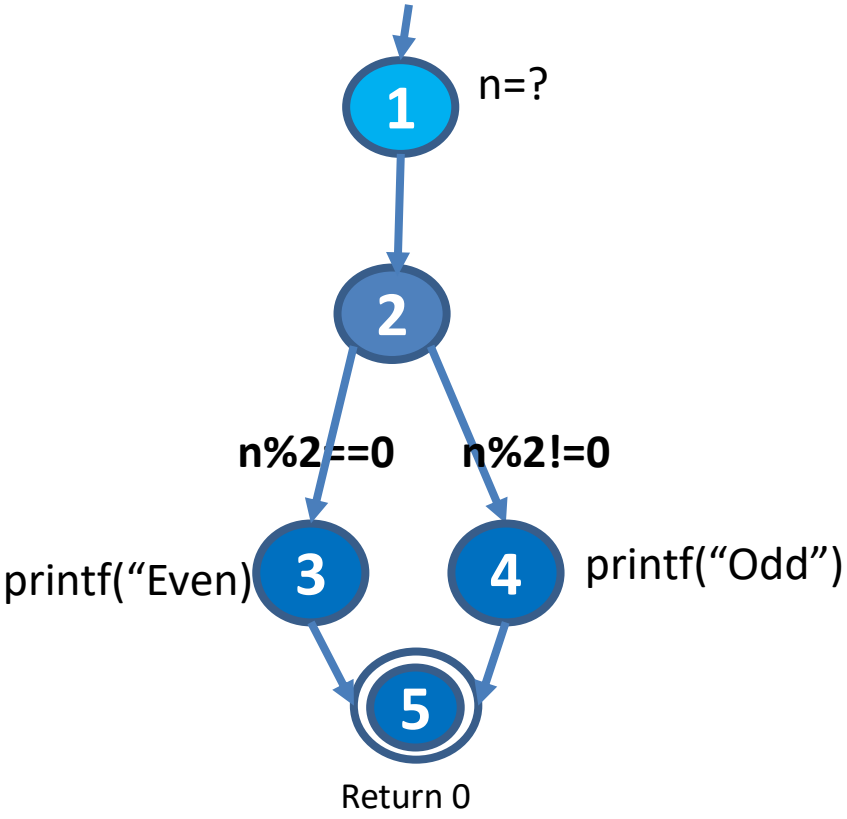
Draw the CFG for the following C program

A program that checks if an integer is odd or even.

```
//Check if a number is odd or even
int oddEven(int n)
{
        if(n%2==0)
                printf("Even")
        else
                printf("Odd");
        return 0;
}
```

CFG for the following C program

```
//Check if a number is odd or even
int oddEven(int n)
{
        if(n%2==0)
           printf("Even");
        else
           printf("Odd");
        return 0;
}
```



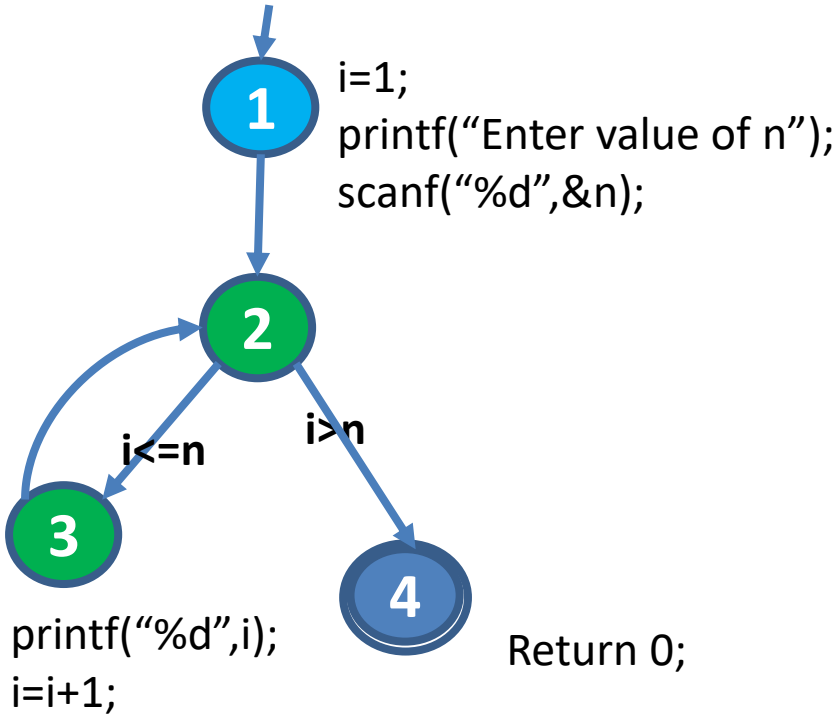
Draw the CFG for the following C program

A program that prints a series of integers.

```
//print 1 to n
int print()
{
        int i;
        printf("Enter value of n");
        scanf("%d",&n);
        for(i=1;i<=n;i++)
                printf("%d",i);
        return 0;
}
```

CFG for the following C program

```
//print 1 to n
int print()
{
        int i=1;
        printf("Enter value of n");
        scanf("%d",&n);
        for(i=1;i<=n;i++)
                printf("%d",i);
        return 0;
}
```



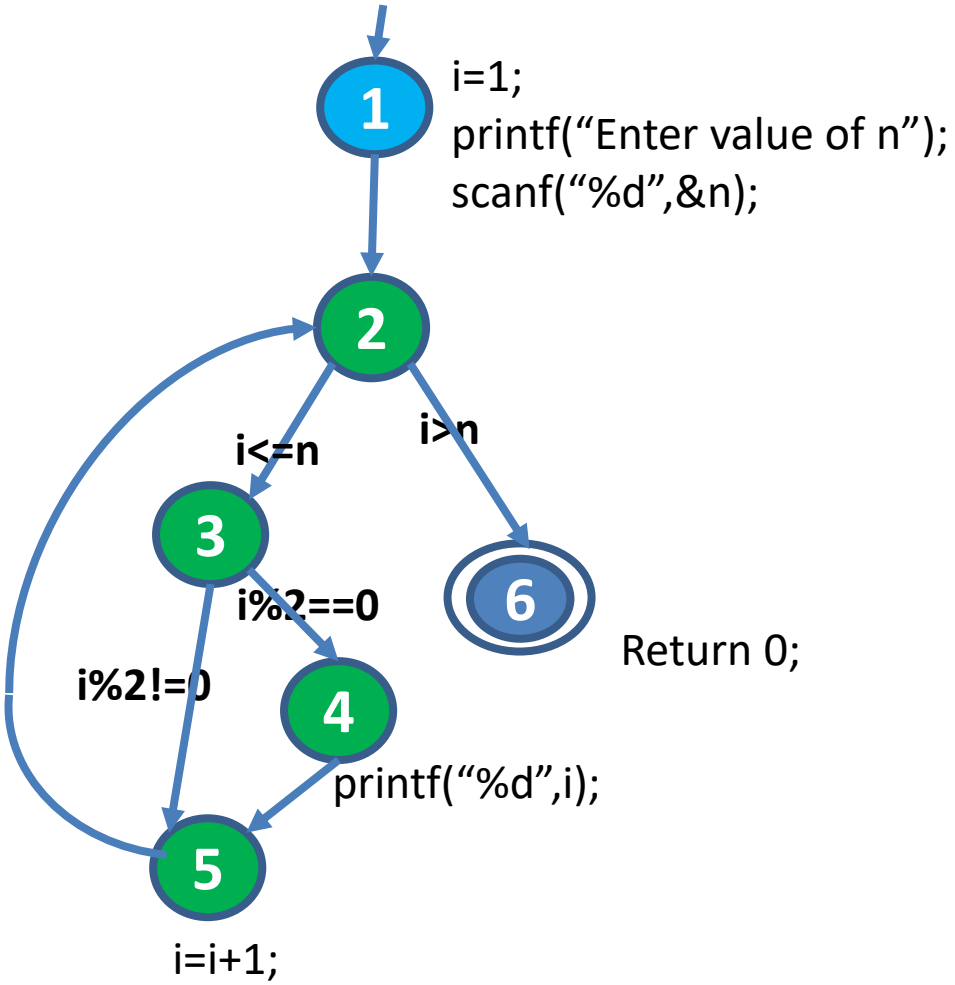
Draw the CFG for the following C program

A program with conditional statement inside a loop.

```
//print 1 to n
int print()
{
        int i;
        printf("Enter value of n");
        scanf("%d",&n);
        for(i=1;i<=n;i++){
            if(i%2==0) printf("%d",i);
        return 0;
}
```

CFG for the following C program

```
//print 1 to n
int print()
{
   int i=1;
   printf("Enter value of n");
   scanf("%d",&n);
   for(i=1;i<=n;i++)
     if(i%2==0)
         printf("%d",i);
   return 0;
}
```



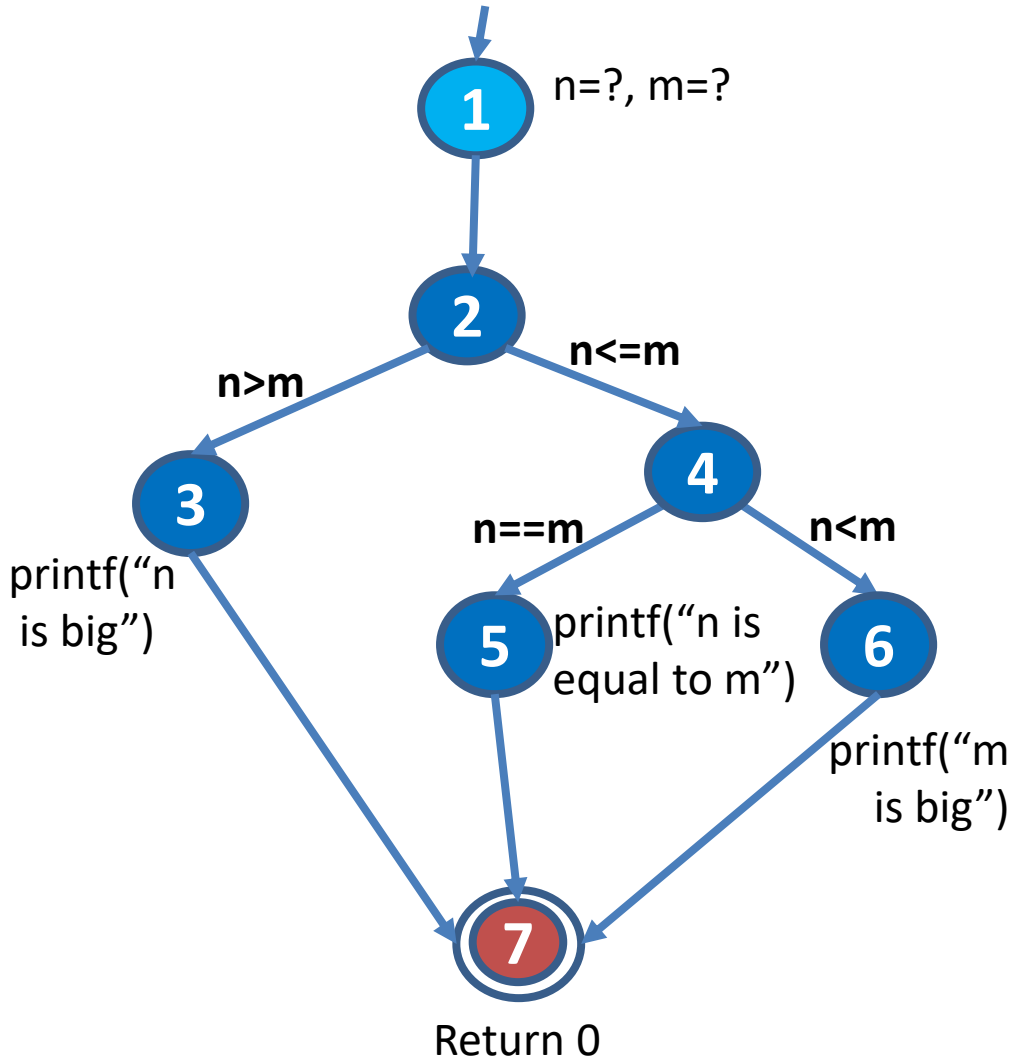
Draw the CFG for the following C program

A program that compares two numbers (if-else if-else ladder like statement).

```
//Compare two numbers
int Compare(int n, int m)
{
  if(n>m)
    printf("n is big")
  else if(n<m)
    printf("m is big");
  else
    printf("n is equal to m");
  return 0;
}
```

CFG for the following C program

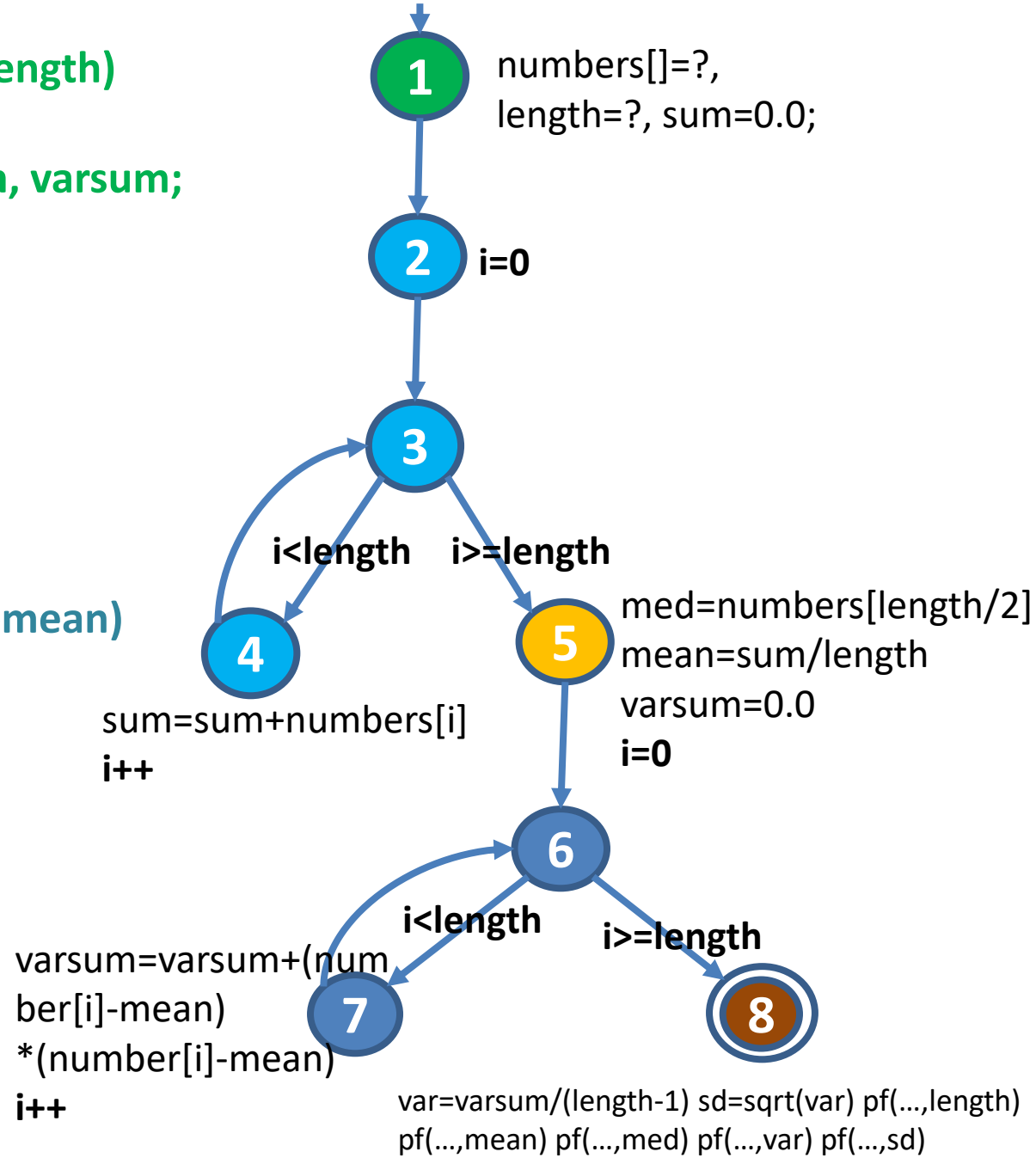
```
//Compare two numbers
int Compare(int n, int m)
{
  if(n>m)
    printf("n is big")
  else if(n<m)
    printf("m is big");
  else
    printf("n is equal to m");
  return 0;
}
```



Another Example of Control Flow Graph

Draw the CFG for the C program

A program that calculates some of the statistical values such as mean, median, variance and standard deviation of a set of sorted numbers stored in a linear array.

```
void stat_fun(int numbers[],int length)
{
  double med, var, sd, mean, sum, varsum;
  sum=0.0;
  for(i=0;i<length;i++)
     sum=sum+numbers[i];
  med=numbers[length/2];
  mean=sum/length;
  varsum=0.0;
  for(i=0;i<length;i++)
      varsum=varsum+(number[i]-mean)
         *(number[i]-mean);
  var=varsum/(length-1);
  sd=sqrt(var);
  printf("Length=%lf",length);
  printf("Mean=%lf",mean);
  printf("Median=%lf",med);
  printf("Variance=%lf",var);
  printf("Std. dev=%lf",sd);
}
```

```
void stat_fun(int numbers[],int length)
{
  double med, var, sd, mean, sum, varsum;
  sum=0.0;
  for(i=0;i<length;i++)
     sum=sum+numbers[i];
  med=numbers[length/2];
  mean=sum/length;
  varsum=0.0;
  for(i=0;i<length;i++)
      varsum=varsum+(number[i]-mean)
         *(number[i]-mean);
  var=varsum/(length-1);
  sd=sqrt(var);
  printf("Length=%lf",length);
  printf("Mean=%lf",mean);
  printf("Median=%lf",med);
  printf("Variance=%lf",var);
  printf("Std. dev=%lf",sd);
}
```